

Task 1

REST API Fundamentals — Leads CRUD

Builds on: None — starting point

Suggested stack: Node.js, Express, PostgreSQL (pg), Postman

Task Summary

Build a plain CRUD REST API over a simplified leads table. No authentication, no advanced features — the goal is fluency with HTTP verbs, status codes, and a clean request/response shape before anything else is layered on.

What to Build

Method	Path	Purpose
GET	/api/leads	List leads (no pagination yet — that's Task 3)
GET	/api/leads/:id	Fetch one lead by id
POST	/api/leads	Create a lead
PUT	/api/leads/:id	Replace a lead
PATCH	/api/leads/:id	Partially update a lead
DELETE	/api/leads/:id	Soft-delete a lead (set deleted = true, never hard-delete)

Requirements

- Use a connection pool (`pg.Pool`), never a single client per request.
- Minimal columns to start: `id`, `full_name`, `age`, `phone_mobile`, `country_code`, `lead_status`, `date_entered`.
- Correct status codes: 200 (read/update ok), 201 (created, with Location header), 204 (deleted, no body), 404 (not found), 400 (malformed request).
- Response body is always JSON, even for errors — no HTML error pages.
- Write a README describing how to run the API locally, including the `.env` variables needed for the DB connection.

Done Looks Like

- Every endpoint above works and returns the right status code, verified with a Postman collection checked into the repo.
- Creating a lead with a missing required field returns 400 with a clear message (full validation rules come in Task 2 — for now, just don't crash).
- Deleting a lead never removes the row from the database.

Task 2

Request Contracts, Validation & Error Handling

Builds on: Task 1

Suggested stack: Node.js, Express, express-validator (or Joi/celebrate)

Task Summary

Turn Task 1's happy-path API into one that behaves predictably when the input is wrong. This is where you learn the full request/response contract: path params, query params, body, response shape, and a real error-handling story.

Standard Response Envelope

Adopt one shape for every response so the Angular side never has to guess:

- Success: { "success": true, "data": ... }
- Error: { "success": false, "error": { "code": "VALIDATION_ERROR", "message": "...", "details": [...] } }

Requirements

- Request parameters: validate that :id in the URL is a positive integer before hitting the DB — reject non-numeric ids with 400, not a raw DB error.
- Query parameters: validate types/ranges up front (e.g. an out-of-range limit) with 400, before they reach SQL.
- Request body: validate required fields, types, string lengths, and enums (e.g. lead_status must be one of High/Medium/Low) using a validation library, not hand-written if-chains.
- Central error-handling middleware: one Express error handler at the bottom of the stack; every route uses an asyncHandler wrapper so a thrown/rejected error always reaches it instead of crashing the process.
- Custom error classes: ValidationError → 400/422, NotFoundError → 404, ConflictError (e.g. duplicate) → 409, AuthError → 401/403.
- Never leak stack traces or raw SQL error text to the client in production mode; log the real error server-side, return a safe message to the client.

Done Looks Like

- Sending a malformed body (wrong type, missing field, bad enum value) always returns 400/422 with a details array listing which fields failed and why.
- An unexpected server error (e.g. DB down) returns 500 with a generic message — the process does not crash and no stack trace reaches the client.
- A request for a non-existent id returns 404, not 200 with null data.

Task 3

Pagination, Sorting & Filtering at Scale

Builds on: Task 2

Suggested stack: Node.js, Express, PostgreSQL, pgAdmin / EXPLAIN ANALYZE

Task Summary

The real leads table can hold 10,000,000+ rows. GET /api/leads can never again return “all rows”, and a naive OFFSET/LIMIT gets slower the deeper you page. Build a listing endpoint that stays fast at that scale.

Requirements

Filtering

- Support combinable filters:
?status=Pending&branch_code=BLR01&campaign_id=123&date_from=2026-08-01&date_to=2026-08-18.
- Every filterable column must have a matching index — show the EXPLAIN ANALYZE output before and after adding the index in your README.

Sorting

- ?sort=date_entered&order=DESC — whitelist sortable columns server-side, never interpolate the column name from user input directly into SQL.

Pagination — two flavors

- Page-based (fine for small/medium result sets): ?page=1&limit=50.
- Cursor/keyset-based (required for deep paging on the full table):
?after=<opaque_cursor>&limit=50, using the sort column + id as the keyset instead of OFFSET.
- Document in the README: why OFFSET 5000000 LIMIT 50 is slow, and why keyset pagination avoids that cost.
- Do not run a COUNT(*) on the full table on every request; either cache/estimate the total or omit “total count” from the cursor-based response entirely.

Done Looks Like

- Seed at least 1,000,000 rows and show p95 latency under ~300ms for a filtered, sorted, paginated request, with EXPLAIN ANALYZE proving index usage.
- Paging to “page 100,000” of the offset endpoint does not take multiple seconds — or the cursor endpoint is used instead and documented as the recommended approach.
- Invalid sort column names are rejected (400), never passed straight into a query string.

Task 4

Authentication API (JWT)

Builds on: Task 1–3 (this task secures those endpoints)

Suggested stack: Node.js, Express, bcrypt, jsonwebtoken

Task Summary

Every endpoint built so far is currently open to anyone. Add login, token issuing, and a middleware that every leads route will require going forward.

Endpoints

Method	Path	Purpose
POST	/api/auth/login	Verify credentials, issue access + refresh tokens
POST	/api/auth/refresh	Exchange a valid refresh token for a new access token
POST	/api/auth/logout	Invalidate the current refresh token

Requirements

- Passwords hashed with bcrypt — never stored or returned in plaintext, and never included in any response body.
- Access token: short-lived (~15 min), carries userId, role, teamId. Refresh token: longer-lived (~7 days), stored server-side (DB or Redis) so it can be revoked on logout.
- requireAuth middleware: verifies the JWT, attaches req.user, rejects missing/expired/invalid tokens with 401.
- requireRole(['admin', 'team_lead']) middleware for role-gated routes, returning 403 for a valid-but-unauthorized user.
- Apply requireAuth to every /api/leads/* route built in Tasks 1–3.

Done Looks Like

- An expired or tampered access token is rejected with 401 on every protected route.
- Logging out invalidates the refresh token — it cannot be used again afterward.
- No response, ever, contains a password hash.

Task 5

Reusable Search: Same API, Debounced & Cached

Builds on: Task 4

Suggested stack: Angular (RxJS), Node.js, an in-memory or Redis cache

Task Summary

The Angular team needs a lead search box. Do not build a second search endpoint — reuse the authenticated GET `/api/leads?q=...` endpoint from Tasks 1–4, and make sure it is not hammered on every keystroke.

Frontend Requirements (Angular)

- Bind the search input to an RxJS Subject/`FormControl.valueChanges`, and pipe through `debounceTime(300)`, `distinctUntilChanged()`, and `switchMap()` before calling the API.
- `switchMap` (not `mergeMap`) so an in-flight request for a stale keystroke is cancelled when a newer one starts.
- Keep a small client-side cache (a Map keyed by the full query string) for the current session so re-typing an already-seen search doesn't re-hit the network.

Backend Requirements (Node.js)

- Add `?q=<text>` to the existing `/api/leads` endpoint — do not create `/api/leads/search` as a separate implementation.
- Add a short-lived server-side cache (5–10 seconds is enough) keyed by the full `filter+query+page` combination, so repeated identical requests from many users don't all hit PostgreSQL.
- Invalidate/bypass the cache for a lead that was just created, updated, or imported so stale data isn't served right after a change.

Done Looks Like

- Typing a search term quickly in the browser's Network tab shows one request per pause in typing, not one per keystroke.
- Repeating an identical search within the cache window does not generate a new database query (log this to prove it).

Task 6

Field-Level Data Masking & Click-to-Call

Builds on: Task 4, 5

Suggested stack: Node.js, Express, PostgreSQL

Task Summary

Sensitive lead fields (phone_mobile, loan_account_no, amount_due, emi_amount, full_country_code+number) must never be exposed to the browser in a list or detail view — not even to Admin. The only exception is a dedicated “click to call” action, which must be logged.

Requirements

- GET /api/leads and GET /api/leads/:id return only: id, full_name, age, gender, primary_address (state-level, not full address), lead_status. This rule applies to every role — Admin and Team Lead included.
- Add POST /api/leads/:id/call. On the server, look up the real phone_mobile + country_code, and either (a) initiate the call via a telephony/dialer API so the raw number is never sent to the browser at all, or (b) return the number for that single action only, never persisted client-side. Pick one approach and justify it in the README — option (a) is the stronger answer.
- Create an audit table: call_actions(id, lead_id, user_id, action, created_at). Every call action writes a row here, no exceptions.
- Rate-limit /api/leads/:id/call per user (e.g. 30/minute) to prevent scraping numbers one call at a time.

Done Looks Like

- Inspecting any /api/leads response in the browser's Network tab never reveals phone_mobile, loan_account_no, amount_due, or emi_amount.
- Every click-to-call action produces a row in call_actions with the correct user and lead.
- Rapid-fire calling the same endpoint past the rate limit returns 429.

Task 7

Bulk CSV / Excel Import at Scale

Builds on: Task 6 (schema, validation rules, RBAC)

Suggested stack: Node.js, csv-parse or exceljs (streaming), PostgreSQL COPY / batch insert, a job queue (BullMQ/Redis or a simple in-process queue)

Task Summary

Admins and Team Leads need to import lead files of 50,000 / 100,000 / 150,000 / 200,000+ rows. A naive “parse the whole file into memory, then loop INSERT one row at a time” approach will time out the HTTP request and can crash the Node process or exhaust the DB connection pool — this task exists to avoid exactly that.

Validation Rules (apply per row, before insert)

Column	Rule
phone_mobile	Required. Must match a valid mobile number pattern.
country_code	Required. Must be a recognized dialing/ISO code.
full_name	Required, non-empty after trimming.
loan_account_no	Required and unique — duplicate values are rejected, not overwritten silently.
next_emi_date_dom	Required, must parse as a valid date.
emi_amount	Required, numeric, greater than 0.
branch_code	Required, must exist in the known branch list.
lead_status	Required, must be exactly one of: High, Medium, Low.

Requirements

- Stream-parse the uploaded file row by row — never load the entire file into memory as one array/object.
- Process the import as a background job: the upload endpoint responds 202 Accepted with a jobId immediately; the actual parsing/inserting happens off the request thread.
- Insert in batches (e.g. 500–1000 rows per multi-row INSERT, or PostgreSQL's COPY) instead of one INSERT per row.
- GET /api/imports/:jobId/status returns { total, processed, succeeded, failed, errors: [...] } so progress can be checked.
- Invalid rows are collected into a downloadable error report (row number + reason) — never silently dropped and never allowed to abort the entire import.

Done Looks Like

- A 200,000-row CSV import completes successfully without the Node process crashing or the DB pool being exhausted (verify pool usage stays within its configured max, see Task 9).
- A file with some invalid rows imports all valid rows and produces a clear, downloadable list of the rejected ones with reasons.
- Re-importing a file with a loan_account_no that already exists rejects/flags those rows instead of creating duplicates.

Task 8

Generic File Upload Service

Builds on: Task 7 (streaming/validation patterns)

Suggested stack: Node.js, multer or busboy, file-type magic-byte checking, local disk or S3-compatible storage

Task Summary

Beyond lead CSVs, the app needs a general-purpose upload endpoint for Excel sheets, images (agent photos, ID proofs), and documents (PDF/DOC). Build one reusable service rather than one-off upload code per feature.

Requirements

- POST /api/files (multipart/form-data) accepting csv, xlsx, png/jpg, pdf/doc.
- Validate by real file signature (magic bytes), not just the extension or the browser-supplied MIME type — a renamed .exe pretending to be a .png must be rejected.
- Enforce a max size per file type (e.g. images 5MB, documents 20MB, CSV/XLSX larger, matching Task 7's needs).
- Store with a generated, non-guessable key (not the original filename) and return { fileId, url, type, size }.
- Uploaded files are not publicly listable/enumerable — access requires the fileId and, where relevant, the same auth/RBAC rules as the rest of the API.

Done Looks Like

- A file with a mismatched extension/signature is rejected with 415.
- An oversized file is rejected with 413 before it's fully written to storage.
- Two uploads of files with the same original filename never collide or overwrite each other.